TLP:AMBER

HHS Cyber

HTOC

HEALTHCARE THREAT OPERATIONS CENTER

HTOCThreatConnect — Lightweight ThreatConnect Client Package

Version 1.0
April 2026

Table of Contents

Table of Contents

1. Purpose

The **HTOCThreatConnect** folder is a small, installable Python package that provides a **read-only (GET-only)** ThreatConnect HTTP client plus two high-level helpers that return indicator data as a **pandas DataFrame**:

`get_v3_threatconnect_data` — Queries ThreatConnect **v3** `/v3/indicators` with TQL, paginates results, and normalizes JSON into a flat table.

`get_v2_threatconnect_data` — Calls ThreatConnect **v2** `/v2/indicators/observed` for a single calendar day of observed indicators per owner.

The package is intended for **notebooks, scripts, and internal tooling** that need a minimal dependency surface (requests + pandas + pytz) without pulling in the full official ThreatConnect SDK.

-

2. Scope

This procedure applies to developers and analysts who install, configure, upgrade, or troubleshoot the package from the repository at `notebooks/HTOCThreatConnect/`, or who consume it from a distributed wheel on the `Z:\` drive.

-

3. Repository Layout

Path	Role
\`HTOCThreatConnect/\`	Main package source (\`ThreatConnect.py\`, \`RequestObject.py\`, \`__init__.py\`)
\`HTOCThreatConnect/utils/config.json\`	API credentials and base URL (see Section 5)
\`HTOCThreatConnect/utils/config_loader.py\`	Loads and validates \`config.json\`
\`HTOCThreatConnect/tc.conf\`	Packaged with the wheel (see \`MANIFEST.in\` / \`pyproject.toml\` package-data)
\`test.ipynb\`	Smoke-test notebook importing \`get_v3_threatconnect_data\`
\`pyproject.toml\`	Build metadata, version, dependencies
\`requirements.txt\`	Minimal \`requests\` pin (full deps declared in \`pyproject.toml\`)
\`build/\`	Generated by \`pip install\` / build — safe to delete and regenerate

-

4. Prerequisites

Requirement	Notes
Python	3.9 or newer (\`requires-python\` in \`pyproject.toml\`)
Network	Reachability to the ThreatConnect API host defined in \`api_base_url\`

Requirement	Notes
Credentials	Valid `api_access_id` and `api_secret_key` in `config.json`
Optional	`Z:\` access if installing from a pre-built wheel under `Z:\wheels\`

Example dependency install command:

```
pip install requests
```

-

5. Configuration

5.1 `config.json`

The helpers load credentials from:

```
notebooks/HTOCThreatConnect/HTOCThreatConnect/utils/config.json
```

When the package is installed, the same path is resolved relative to the installed package directory (`Path(__file__).parent / "utils" / "config.json"`).

Required keys (validated by `load_config`):

Key	Description
`api_secret_key`	ThreatConnect API secret key
`api_access_id`	ThreatConnect API access ID
`api_base_url`	API root URL (e.g. `https://<host>/api`)

Key	Description
`api_default_org`	Default organization name (must match playbook/query expectations; code uses `HTOC Org` in TQL)

If any key is missing or empty, `load_config` raises a `KeyError`.

5.2 Secrets handling

Do not commit real credentials to git. Use a local `config.json` that is excluded from version control, or inject credentials via your deployment pattern.

Rotate keys in ThreatConnect if a file is ever exposed.

Treat `config.json` like a password file: least-privilege file permissions on disk.

-

6. Installation

6.1 Editable install from the repository (development)

From the package root ([`notebooks/HTOCThreatConnect/`](notebooks/HTOCThreatConnect/)):

```
cd H:\HTOC\notebooks\HTOCThreatConnect
py -m pip install -e .
```

This allows editing source files and immediately using the updated package in Python or Jupyter.

6.2 Install from a wheel (distribution)

Per `README.md`, example:

```
py -m pip install Z:\wheels\HTOC_threatconnect-0.1.10-py3-none-any.whl
```

Upgrade an existing install:

```
py -m pip install --upgrade Z:\wheels\HTOC_threatconnect-0.1.10-py3-none-any.whl
```

Bump the version in `pyproject.toml` before building a new wheel so consumers can tell releases apart.

6.3 Build a wheel (maintainers)

From the package root, using standard setuptools:

```
cd H:\HTOC\notebooks\HTOCThreatConnect
py -m pip install --upgrade build
py -m build
```

Copy the generated `.whl` from `dist/` to `Z:\wheels\` (or your internal artifact location) and document the version in release notes.

-

7. API Reference (High-Level)

7.1 `get_v3_threatconnect_data`

```
from HTOCThreatConnect import get_v3_threatconnect_data
```

```
df = get_v3_threatconnect_data()                          # default lastObserved >= 2023-01-01
df = get_v3_threatconnect_data(lastObserved_date="2025-01-01")
```

Parameter	Type	Default	Description
`lastObserved_date`	`None`, `str` (`YYYY-MM-DD`), `datetime.date`, or `datetime.datetime`	`None` → `"2023-01-01"`	Lower bound for `lastObserved` in TQL (`00:00:00Z` on that day)
`indicatorActive`	`bool`	`True`	When `True`, TQL includes `indicatorActive=true`. When `False`, inactive only.
`bothActivities`	`bool`	`False`	When `True`, TQL uses `(indicatorActive EQ true OR indicatorActive EQ false)`

Hardcoded behavior (in source):

Owner: `HTOC Org` only (list `OWNERS` in `ThreatConnect.py`).

Indicator types: Address, Email Address, File, Host, URL, ASN, CIDR, Email Subject, Hashtag, Mutex, Registry Key, Stripped URL, User Agent.

Endpoint: `GET /v3/indicators` with `fields=tags,associatedGroups`.

Pagination: `resultLimit=10000`, advances `resultStart` until a short page.

SSL verification: Disabled (`VERIFY_SSL = False`); urllib3 insecure-request warnings suppressed.

Output: DataFrame with nested fields flattened via `pandas.json_normalize`; deduplicated on derived `indicator` column; several redundant columns dropped after normalization.

7.2 `get_v2_threatconnect_data`

```
from HTOCThreatConnect import get_v2_threatconnect_data

df = get_v2_threatconnect_data()                   # today's date (UTC) on v2 observed API
df = get_v2_threatconnect_data(dateObserved="2025-10-01")
```

Parameter	Type	Default	Description
`dateObserved`	`None`, `str`, `date`, or `datetime`	`None`	Calendar day passed as `dateObserved=YYYY-MM-DD` to v2 observed endpoint

Hardcoded behavior:

Owner: `HTOC Org`.

Endpoint: `GET /v2/indicators/observed?owner=...&dateObserved=...`

No pagination loop in code — assumes one response carries the full set for that day.

SSL verification: Disabled, same as v3 helper.

-

8. `ThreatConnect` Class (Low-Level)

Exported from `HTOCThreatConnect.ThreatConnect`:

`read_only = True` — Any HTTP method other than **GET** raises `PermissionError`.

Authentication: HMAC (`TC {accessId}:{signature}`) using `api_aid` / `api_sec` as passed into the constructor.

Retries: Up to 5 attempts with ~59 s sleep on read timeout or connection errors.

`set_verify_ssl(bool)` — Toggle TLS verification (helpers set it to `False` today).

Use this class only when you need custom GET paths beyond the two helpers; keep writes in separate code paths that use the official SDK or ThreatConnect UI.

-

9. Smoke Test (`test.ipynb`)

The notebook `test.ipynb` demonstrates:

```
from HTOCThreatConnect import get_v3_threatconnect_data
```

Running the helper and inspecting the resulting DataFrame.

Verify after install or config change:

Open `test.ipynb` in Jupyter or VS Code.

Select the kernel that has the package installed (same environment as `pip install`).

Run all cells.

Confirm a non-empty DataFrame when the instance has matching data for the chosen `lastObserved_date`; an empty DataFrame is valid if no rows match the TQL window.

-

10. Versioning and Release Checklist

Update **`version`** in `pyproject.toml` (and align wheel filename / README examples).

Run smoke tests (`test.ipynb` or a short script calling both helpers).

Build wheel (`py -m build`), publish to `Z:\wheels\` or internal store.

Notify consumers to `pip install --upgrade` with the new wheel path.

Never ship a wheel that embeds real `config.json` secrets — consumers should supply their own config or environment-specific packaging.

-

11. Troubleshooting

Symptom	Likely cause	Resolution
`FileNotFoundError: Config file not found`	`config.json` missing next to installed package	Create `HTOCThreatConnect/utils/config.json` with required keys; reinstall if using wheel without package-data
`KeyError` from `load_config`	Missing or null key in JSON	Fill all four required keys
`ModuleNotFoundError: HTOCThreatConnect`	Package not installed in active kernel	`pip install -e .` from repo root or install wheel into that environment
`PermissionError: Read-only client`	Code attempted non-GET	Use official SDK or HTTP client for writes; this package is GET-only by design
`RuntimeError` with binary response body	Non-2xx API response	Check URL, credentials, owner name, and TQL; decode error message from ThreatConnect
Empty DataFrame from v3 helper	No indicators in window for `HTOC Org`	Widen `lastObserved_date` or confirm data exists in ThreatConnect UI

Symptom	Likely cause	Resolution
Empty DataFrame from v2 helper	No observed rows for that UTC date	Confirm v2 observed API returns data for the date in UI or Postman
SSL / proxy errors	Local network or `VERIFY_SSL=False` mismatch with corporate policy	Work with network team; consider `set_verify_ssl(True)` if CA trust is available

-

12. Relationship to Other HTOC Components

Tipper, PRISM notebooks, and other automation may use either this package or the full ThreatConnect SDK under `\\10.1.4.22\...\threatconnect` — standardize on one client per project to avoid duplicate credential files.

SOPs for playbooks (`SOP_TC_Playbook_*.md`) describe ThreatConnect automation in the UI; this package is for **Python-side** read access to the same platform.

-

13. Related Documents

`notebooks/HTOCThreatConnect/README.md` — quick install and usage

`SOP_PRISM_ThreatAssessmentScoring.md` — notebook that also integrates ThreatConnect (different client path)

`SOP_Tipper_DailyIndicatorTips.md` — batch script using ThreatConnect SDK

ThreatConnect API v3 indicators: [ThreatConnect REST API](https://docs.threatconnect.com)